

Aim:- Develop understanding to developing method, method overriding & overloading.

Theory

1. Methods in Java:- method is a block of code that performs a specific task.

It is executed when it is called.

Method helps in:

- Reusability of code.
- Reducing program complexity
- Improving readability.

Syntax of method:-

```
returntype methodname(parameters)
{
    // method body
}
```

Ex

```
void display()
```

```
{
    System.out.println("Hello Java");
}
```

2. Method overloading:- Method overloading occurs when multiple methods have the same name but different parameters within the same class.

It is an example of compile-time polymorphism.

Rules of method overloading-

- method name must be the same
- Parameter list must be different
- Return type may or may not be different.

Example

```

class overloading {
    void add(int a, int b) {
        System.out.println("Sum : " + (a+b));
    }
    void add(int a, int b, int c) {
        System.out.println("Sum : " + (a+b+c));
    }
    public static void main(String[] args) {
        overloading obj = new overloading();
        obj.add(10, 20);
        obj.add(10, 20, 30);
    }
}

```

output

sum : 30

sum : 60

3. Method overriding :- method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its parent class. It is also example of runtime polymorphism.

Rules of method overriding-

- Method name must be the same.
- Parameters must be the same
- Must occur in inheritance
- Return type must be same or compatible.

Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class Overriding {  
    public static void main (String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output

Dog barks.

Program Demonstrating method

```

Class MethodDemo {

```

```

    void display() {

```

```

        System.out.println("This is a simple
                           method");
    }

```

```

}

```

```

public static void main (String[] args) {

```

```

    MethodDemo obj = new MethodDemo();

```

```

    obj.display();

```

```

}

```

```

}

```

Difference between method overloading & overriding

Feature	Method overloading	Method overriding
Definition	Same method name with different parameters	Same method name & parameters
Class	within same class	parent & child class
Polymorphism	compile time	run time
Inheritance	Not required	Required

Result

Thus, Java methods, method overloading, method overriding were studied & implemented successfully.

output

This is a simple
method.

Aim:- Write object oriented program in Java
: Access modifier, class & object.

Theory Object oriented programming is a programming paradigm that organizes software design around objects & classes instead of functions & logics.

Main OOP concept

- Class
- Object
- Encapsulation
- Abstraction
- Inheritance
- Abstraction

Class:- A class is a blueprint or template used to create object.

It defines properties (variable) & behaviour (method).

Syntax class student
 {
 int id;
 String name;
 }

- Acts as a template to create object with shared structure.
- It doesn't occupy memory for fields until instantiation.
- It can contain field, method, constructors, nested classes & interfaces.

Teacher's Signature _____

Its characteristics -

State → Value stored in field.

Behaviour → Actions defined through methods

Identify → Distinguishes one object from another.

Object:- An object is instance of a class.

Each object holds its own state.

It is a runtime entity created using "new" keyword, which occupies memory & contains actual values.

^{Note} 'new' keyword allocate memory for the object in heap memory.

Constructor used to initialize the object.

Ex Student s1 = new Student ();

↓ ↓ ↓
 class name reference Constructor
 & variable

Note s1 is not object name. it is a reference.

→ object is created in heap memory.

→ Constructor is automatically called.

Constructor:- special method that is automatically called when object is created.

Properties

→ Same Name as Class

→ No return type

→ Used to initialize object.


```

Ex student 1) {
    System.out.println("Constructor called");
}

```

Access Specifiers :-

Access specifier defines accessibility of a variable & methods.

Access Specifier	Same Class	Same Package	Sub-class	Other package
Private	✓	x	x	x
Default	✓	x	x	x
Protected	✓	✓	✓	x
Public	✓	✓	✓	✓

Program 1 Create an instance of class student allocates memory using 'new'. Calls the constructor & assign its reference to s1

```

class Student {
    int id;
    String Name;
    void Display() {
        System.out.println("Id:" + id);
        System.out.println("Name: " + Name);
    }
}

```

```
public class mainStudent {  
    public static void main( String[] args) {  
        Student s1 = new Student();  
        s1.id = 103;  
        s1.name = "Keechan";  
        s1.Display();  
    }  
}
```

Program 2 Demonstration of Access Specifiers

```
class DefaultClass {
```

```
    int defaultVar = 10;
```

```
    void display() {
```

```
        System.out.println("This is default class method");  
    }
```

```
}
```

```
public class AccessSpecifierDemo {
```

```
    private int privateVar = 20;
```

```
    protected int protectedVar = 30;
```

```
    public int publicVar = 40;
```

```
    private void privateMethod() {
```

```
        System.out.println("This is private method");  
    }
```

```
    protected void protectedMethod() {
```

```
        System.out.println("This is protected method");  
    }
```



```
public void publicMethod() {
    System.out.println("This is public method");
}
```

```
public static void main(String[] args) {
    AccessSpecifierDemo obj = new AccessSpecifierDemo();
}
```

```
System.out.println("Private variable is: " +
    obj.privateVar);
```

```
System.out.println("Protected variable is: " + obj.protectedVar);
```

```
System.out.println("Public variable is: " + obj.publicVar);
```

```
obj.privateMethod();
```

```
obj.protectedMethod();
```

```
obj.publicMethod();
```

```
DefaultClass d = new DefaultClass();
```

```
System.out.println("Default Variables: " +
    d.defaultVar);
```

```
d.display();
```

```
}
```

```
}
```

Program 3 Demonstration of multiple methods in Different classes.

```
Class Text {
```

```
    public static void E() {
```

```
        int a = 5;
```

```
        System.out.println("In method E & value is: " + a);
```

```
    }
```

```
}
```

Teacher's Signature _____

```
public class MethodsDemo {  
    public static void D() {  
        int a = 5;  
        System.out.println("In method D the value  
            is: " + a);  
    }  
}
```

```
    public static void C() {  
        float b = 2;  
        System.out.println("In method C the value  
            is: " + b);  
    }  
}
```

```
    public static void B() {  
        int a = 5; float  
        C();  
        System.out.println("In method B the value is: " + a);  
    }  
}
```

```
    public static void A() {  
        int a = 3;  
        C();  
        System.out.println("In method A the value is: " + a);  
    }  
}
```

```
    public static void main (String[] args) {  
        A();  
        D();  
        Text.E();  
    }  
}
```